

Design and Analysis of Algorithms

2nd Sessional Exam - Detailed Solutions

1. Explain Greedy Algorithm

Definition:

A Greedy Algorithm is an algorithmic paradigm that builds up a solution piece by piece, always choosing the next piece that offers the most obvious and immediate benefit. It makes the locally optimal choice at each stage with the hope that these local choices will lead to a globally optimal solution.

Key Characteristics:

For a problem to be optimally solved using a greedy approach, it must exhibit two main properties:

1. **Greedy-Choice Property:** A global optimum can be arrived at by selecting a local optimum. You can make whatever choice seems best at the moment and then solve the subproblems that arise later, without reconsidering past choices.
2. **Optimal Substructure:** An optimal solution to the problem contains within it optimal solutions to subproblems.

Components of a Greedy Algorithm:

- **Candidate Set:** The set of items from which a solution is created.
- **Selection Function:** Chooses the best candidate to be added to the solution.
- **Feasibility Function:** Checks if a candidate can be used to contribute to a valid solution (without violating constraints).
- **Objective Function:** Assigns a value to a solution or a partial solution.
- **Solution Function:** Indicates when we have discovered a complete solution.

Pros and Cons:

- **Advantages:** They are generally easy to invent, easy to implement, and highly efficient (often linear $O(n)$ or $O(n \log n)$ time complexity).
- **Disadvantages:** They do not always yield the globally optimal solution. If a problem does not strictly have the greedy-choice property, a greedy algorithm will fail to find the best result.

Standard Examples:

- Fractional Knapsack Problem
- Dijkstra's Shortest Path Algorithm
- Prim's and Kruskal's Minimum Spanning Tree Algorithms
- Huffman Coding

2. Explain Dijkstra's Algorithm

Definition:

Dijkstra's Algorithm is a popular greedy algorithm used to find the shortest path from a single source vertex to all other vertices in a given weighted graph.

Condition:

The graph can be directed or undirected, but it **must have non-negative edge weights**. If there are negative weights, the algorithm may fail to find the correct shortest path (Bellman-Ford is used instead).

Working Principle:

The algorithm maintains two sets of vertices:

1. A set of vertices whose shortest distance from the source is already finalized.
2. A set of unvisited vertices whose shortest distance is not yet finalized.

At every step, the algorithm selects the unvisited vertex that has the minimum distance from the source, finalizing its distance, and then "relaxes" (updates) the distances of all its adjacent neighboring vertices.

Algorithm Steps:

1. Initialize the distance to the source node as **0** and the distances to all other nodes as infinity (∞).
2. Mark all nodes as unvisited. Create an empty set (or Priority Queue) to keep track of unvisited nodes.
3. While there are unvisited nodes:
 - Select the unvisited node u with the smallest known distance.
 - Mark u as visited (its shortest path is now finalized).
 - For each unvisited neighbor v of u , calculate the tentative distance: **$dist(u) + weight(u, v)$** .
 - If this tentative distance is less than the currently recorded distance of v , update the distance of v to this new, lower value.

Time Complexity:

- Using a simple array/matrix: **$O(V^2)$** where **V** is the number of vertices.
- Using an Adjacency List and a Min-Priority Queue (Min-Heap): **$O((V + E) \log V)$** where **E** is the number of edges.

3. Explain Knapsack Problem

Definition:

The Knapsack Problem is a classic combinatorial optimization problem. Given a set of items, each with a specific weight and a value, the objective is to determine the number of each item to include in a "knapsack" (which has a maximum weight capacity) so that the total weight is less than or equal to the capacity, and the total value is maximized.

Types of Knapsack Problems:

There are two primary variations of this problem in algorithm design:

1. Fractional Knapsack Problem:

- *Concept:* The items can be broken down into smaller pieces. You can take a fraction of an item if the whole item doesn't fit.
- *Approach:* Solved optimally using a **Greedy Algorithm**.
- *How it works:* Calculate the value-to-weight ratio for each item. Sort the items in descending order of this ratio. Greedily add whole items from the top of the list until the knapsack is full. For the last item that exceeds the remaining capacity, take the exact fraction that fits.

2. 0/1 Knapsack Problem:

- *Concept:* Items are indivisible. You must either take an entire item (1) or leave it behind (0).
- *Approach:* The greedy approach fails here because taking a high-value item might block you from taking multiple smaller items that together have a higher value. It must be solved using **Dynamic Programming**.
- *How it works:* We build a 2D table (matrix) where rows represent items and columns represent capacities from 0 up to W (max capacity). We recursively calculate the maximum value by checking two cases for each item:
 - a) Including the item (if it fits).
 - b) Excluding the item.

The formula is: $K[i][w] = \max(val[i] + K[i-1][w-wt[i]], K[i-1][w])$

4. Explain Recurrence Relation

Definition:

In computer science, a recurrence relation is a mathematical equation or inequality that defines a function in terms of its values on smaller inputs. It is the primary tool used to analyze the time and space complexity of **recursive algorithms** (particularly Divide and Conquer algorithms).

Structure:

A recurrence relation typically has two parts:

1. **Base Case:** The condition under which the recursion stops (e.g., $T(1) = 1$).
2. **Recursive Step:** The equation that breaks the problem into smaller subproblems.
 - *General Form:* $T(n) = aT(n/b) + f(n)$
 - Where a is the number of subproblems, n/b is the size of each subproblem, and $f(n)$ is the cost of the work done outside the recursive calls (like dividing the problem or merging the results).

Example:

For the Merge Sort algorithm, the array is divided into 2 halves, each recursively sorted, and then merged in linear time. The recurrence relation is:

$$T(n) = 2T(n/2) + O(n)$$

Methods for Solving Recurrence Relations:

To find the Big-O time complexity from a recurrence relation, we solve it using one of three standard methods:

1. **Substitution Method (Iteration Method):** We repeatedly substitute the recurrence into itself to find a mathematical pattern, which we then sum up and optionally prove using mathematical induction.
2. **Recursion Tree Method:** We draw a tree where each node represents the cost of a single subproblem. We sum the costs across each level of the tree, and then sum all the levels to get the total time complexity.
3. **Master Theorem:** A direct, formulaic approach used to solve relations of the specific form $T(n) = aT(n/b) + f(n)$. By comparing $f(n)$ with $n^{\log_b a}$, we can instantly determine the time complexity bound.